SLIIT UNI
THE KNOWLEDGE UNIVERSITY

2025_02

Final Report:
INDOOR ENVIRONMENT MONITORING SYSTEM.

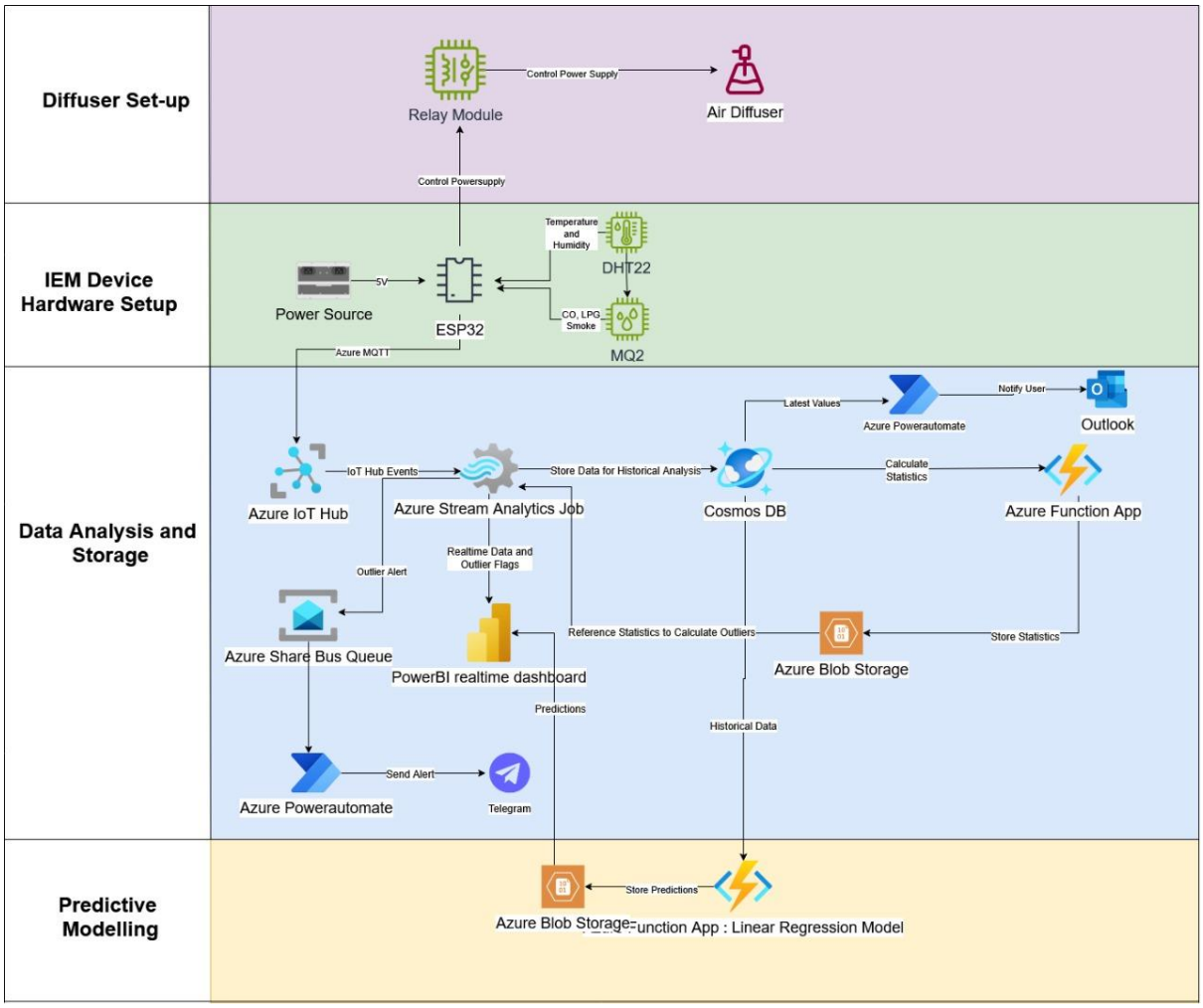
IT4021 : Internet Of Things and Big Data Analytics

Student ID number	Name
IT21174230	Herath H.M.T.N **(Leader)**
IT21380914	Jayawardena K.M.S.P.

1. Project Description.

Indoor Environment Monitoring (IEM) system focuses on implementing an IoT-based environmental monitoring system that collects real-time data on the Smoke, Liquid Petroleum Gas (LPG) and Carbon Monoxide(CO) levels, Humidity and Temperature of the Indoor environment. The collected data is then stored and analyzed, after which the results and predictions of the analysis is displayed in a dashboard making the Results and predictions easily understandable. Alerts for CO. LPG and smoke levels will be displayed in the dashboard. In addition, If the Humidity level in the environment is lower than the optimal level a diffuser will be triggered to on state, which could be manually switched Off through the dashboard or it will automatically switch of based on the humidity level. With all of these functionalities, the system will assist in air quality monitoring and environmental controls and assessment [1] [2] [3].

2. Architecture Diagram and Description



The proposed system is a smart indoor environment monitoring (IEM) system that integrates hardware sensors with Azure cloud services for real-time data processing, visualization, and alerting.

The architecture is composed of three main components which are IEM Device Hardware Setup, Diffuser Setup, and finally Data Analysis and Storage.

Discussing the components of the system starting with the IEM device hardware setup, where at the core of the hardware layer is the ESP32 microcontroller, connected to two key sensors DHT22 used for Measuring temperature and humidity and the MQ2 which detects gases such as carbon monoxide (CO), liquefied petroleum gas (LPG), and smoke.

With the diffuser set up, the diffuser unit is controlled via a relay module, which functions as an electronic switch. Based on sensor readings when there are continuous low-level readings for humidity the ESP32 microcontroller triggers the relay to activate the air diffuser, ensuring timely air purification.

Lastly for the Analysis and visualization component of the system, Sensor data is streamed from the ESP32 to Azure IoT Hub, which acts as the ingestion point for telemetry data. Then through an Azure stream analytics job the real-time and flagged data is sent to power BI Dashboard for live monitoring and the data is also sent to Cosmos DB for purposes of historical data analytics and reference statistics are send to Azure blob storage to support outlier detection.

This system enables environmental monitoring combining low –cost IOT hardware with scalable azure services. It ensures both real time response through diffuser activation and insightful decision making via live dashboards and historical data analytics.

3. Implementation.

3.1. Gas Detection Implementation.

MQ2 Gas Sensor Calibration

MQ2 Sensor is more specialized in detecting CO, LPG, Smoke and other combustible gases than MQ135. However, it needs 24hr burn-n time and calibration to get proper values. In calibration, the sensor is placed in clean air to identify the resistance in clean air. According to the datasheet for MQ2, $\mathrm{Ro} = \mathrm{Rs} / 3.83$, where Ro is the resistance in clean air, Rs is the sensor resistance.

We used 50 samples, 500 ms apart from each other to find Ro. Calculated Ro should be between 0.5 and 1.5 KOhm for clean air. The result of calibration was,

```
Average Rs in clean air: 8.22 KOhms

Calculated Ro: 0.84 KOhms

Calibration is done...

Ro=0.84kohm
```

MQTT Broker and IOT Hub Configuration

Next Azure IoT Hub was used to record events from ESP32. IoT Explorer was used to monitor the telemetry and confirm that data sent by ESP32 are truly send to IoT Hub as events.

Data Storage

Azure Cosmos DB was used as persistent storage for storing events as it is schema-less and simple to use with time series data. The events are recorded in json format. To load events into Azure Cosmos

DB, an Azure Stream Analytics Job was created. The stream analytics job, in addition to saving data to Cosmos DB for historical analysis, it also streamed real time data into a Power BI Streaming Dataset for real time analysis.

Historical Data Gathering

While collecting data, we simulated various possible scenarios such as incense/ mosquito repellent being lit and smoking cigarettes. This is to help the outlier detection logic distinguish between these incidents and an actual gas leakage. We simulated only single instance of gas leakage to avoid references statistics such as mean and standard deviation being influenced by outlier event.

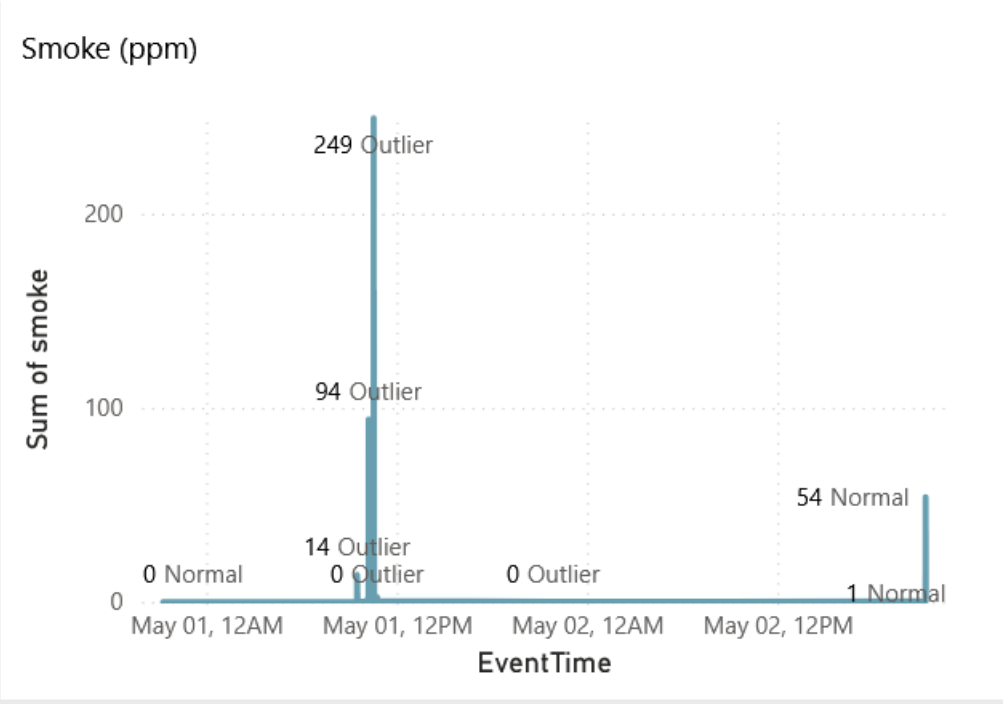


Figure 1: Outliers vs High Values

Data Analysis

Using real time data, we detect outliers in gas sensor values and events where humidity level is below 40%. To detect real time outliers, we used the statistical method, the purpose of this is to identify potential gas leaks. For all gas sensor data, using mean and standard deviation, upper bounds for outliers were introduced. If the value exceeds this upper bound, it can be a possible gas leak. To calculate mean and standard deviation for gas sensor data using historical data, an Azure Function App was created. The function app runs every 2 days to update the values with new data. This way, even if the nature of data constantly kept changing, our outlier detection logic will not be outdated. To store these reference statistics, a separate Azure Blob Storage was used, and it was added as an input to our stream analytics job. Stream analytics also sends a message to a shared Azure Bus Service Queue. Power Automate listens on this queue and triggers an action that sends a telegram alert to the user about a potential gas leak. Given below is the full query of the stream analytics job.

```
SELECT
    EventEnqueuedUtcTime AS id,
```

```
    lpg,
    co,
    smoke,
    temp,
    humid,
    EventProcessedUtcTime,
    PartitionId,
    IoTHub.ConnectionDeviceId AS Air,
    IoTHub.ConnectionDeviceGenerationId,
    IoTHub.EnqueuedTime
INTO
    [data-cont]
FROM
    [IEMDevice];


SELECT
    System.Timestamp() AS EventTime,
    input.EventEnqueuedUtcTime AS id,
    input.temp,
    input.lpg,
    input.co,
    input.smoke,
    input.humid,
    input.EventProcessedUtcTime,
    input.PartitionId,

    ref.mean_co,
    ref.std_co,
    ref.mean_smoke,
    ref.std_smoke,
    ref.mean_lpg,
    ref.std_lpg,

    -- Bounds for CO
    ref.mean_co + 3 * ref.std_co AS co_upper_bound,
    ref.mean_co - 3 * ref.std_co AS co_lower_bound,

    -- Bounds for Smoke
    ref.mean_smoke + 3 * ref.std_smoke AS smoke_upper_bound,
    ref.mean_smoke - 3 * ref.std_smoke AS smoke_lower_bound,

    -- Bounds for LPG
    ref.mean_lpg + 3 * ref.std_lpg AS lpg_upper_bound,
    ref.mean_lpg - 3 * ref.std_lpg AS lpg_lower_bound
```

```
INTO
    [IEM-Workspace]
FROM
    [IEMDevice] AS input
JOIN
    [references] AS ref
ON
    input.IoTHub.ConnectionDeviceId = ref.Air;

SELECT
    CONCAT('{ "chat_id": "8116913151", "text": "🚨 High gas levels detected
(CO, LPG, or Smoke)! ',
           'Please check your environment." }') AS message
INTO
    [alerts-queue]
FROM
    [IEMDevice] AS input
JOIN
    [references] AS ref
ON
    input.IoTHub.ConnectionDeviceId = ref.Air
WHERE
    input.co > ref.mean_co + 3 * ref.std_co OR
    input.smoke > ref.mean_smoke + 3 * ref.std_smoke OR
    input.lpg > ref.mean_lpg + 3 * ref.std_lpg;
```

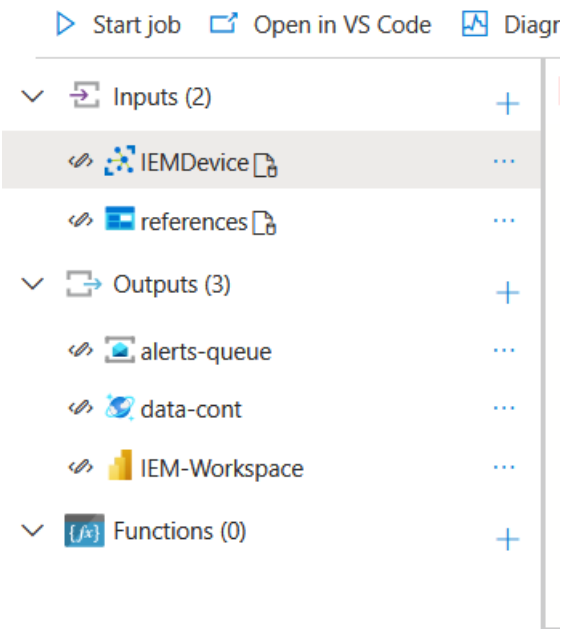


Figure 2: Inputs and Outputs of Stream Analytics

Data Visualization

[SS OF THE DASHBOARD]

3.2. Temperature and Humidity Prediction Implementation.

Implementation of Controls for Diffuser.

The implementation of the diffuser was introduced in the hopes of maintaining the optimal indoor humidity levels, in order to achieve that the ESP32 executes an embedded control logic via Arduino code Were. Each hour, it calculates the average humidity based on the five most recent readings. If the average falls below a threshold of 40% triggers a relay module connected to the ESP32 to power on the diffuser autonomously. Simultaneously, Power Automate is utilized to monitor humidity through readings stored in cosmos DB and with the same logic of the diffuser it sends and emails to the user if the average humidity per hour is below 40%

These readings are then transferred though ad ESP32 to Azure IOT hub enabling seamless real time communications between hardware and cloud services. Azure IOT hub is configured to route the incoming data to power BI and cosmos DB. Where power BI provides near real-time visualization of humidity trends, allowing users to monitor environmental conditions as they evolve. Meanwhile, Cosmos DB ensures long-term data storage, preserving historical sensor readings for future analysis and model training.

Implementation of Temperature Forecasting.

A temperature forecasting mechanism is introduced using Azure Function App. This function hosts a regression-based predictive model that leverages historical temperature data stored in Cosmos DB. The model is designed to analyze patterns in temperature variations and generate forecasts for the next 24 hours. These predictions provide valuable insights into expected temperature trends, enabling users to make informed decisions such as adjusting heating or cooling systems in advance.

The predicted temperature data is visualized through Power BI, creating a comprehensive and user-friendly dashboard that combines both historical data and future trends. This integration allows for a dynamic view of environmental conditions, helping users to anticipate temperature fluctuations and optimize indoor comfort efficiently

Data Storage and Analysis.

Data was collected and stored in Azure Cosmos DB NoSQL Core, where data was Stored in a JSON format. This data was used for generating alerts for the humidifier as well as the base training dataset of the 24-hour temperature prediction model.

This model was trained and used to generate the Upcoming values for Temperature Based on the sensor readings. To achieve this Azure function app service was used where a python based application was used to train the model from Azure Cosmos DB based data and to Store the predicted output in an Azure Blob Storage container in CSV format.

The predicted data stored in Blob Storage was then integrated into power BI. Since the predictions are 5 times per hour when displaying the expected temperature as a scatter plot, we take the aggregated average of all the temperature values predicted for a given hour. This was done to make the predictions as accurate as possible. The following code is used inside the Azure Function to predict temperature using Linear Regression

```
import logging
```

```
import azure.functions as func
import pandas as pd
from datetime import datetime, timedelta
from sklearn.linear_model import LinearRegression
import numpy as np
from azure.cosmos import CosmosClient, PartitionKey
from azure.storage.blob import BlobServiceClient, BlobClient, ContainerClient

# Cosmos DB setup
COSMOS_URL = 'https://iotprojectdb.documents.azure.com:443/'
COSMOS_KEY =
'k60e0ezGPsTl0mwJDbtQSLOBqGwZ40YGIxI1R7mxp6NOxj53kayj61P5kon28XhCE6Za0qeWpv6cA
CDbiOK6aA=='

DATABASE_NAME = 'iotdatabase'
CONTAINER_NAME = 'sensordata'

# Blob Storage setup
BLOB_CONN_STR =
'DefaultEndpointsProtocol=https;AccountName=iotesp32;AccountKey=d7hdT+m3OgKQIH
j9cgtpbwf8p0LYqAzxCsMhRSaGXVIKzQ50POmiOu6qwBVB28H27UCcbaprTxIo+AStuthUZg==;End
pointSuffix=core.windows.net'  # Replace with your Blob Storage connection
string
BLOB_CONTAINER_NAME = 'predictions'  # The name of your container

# Connect to Cosmos DB
client = CosmosClient(COSMOS_URL, COSMOS_KEY)
database = client.get_database_client(DATABASE_NAME)
container = database.get_container_client(CONTAINER_NAME)

# Connect to Blob Storage
blob_service_client = BlobServiceClient.from_connection_string(BLOB_CONN_STR)
container_client =
blob_service_client.get_container_client(BLOB_CONTAINER_NAME)

def run_prediction(data):
    df = pd.DataFrame(data)
    if 'timestamp' not in df or 'temperature' not in df:
        logging.warning("Missing required fields")
        return None

    df['timestamp'] = pd.to_datetime(df['timestamp'])
    df['temperature'] = df['temperature'].astype(float)
    df = df.sort_values(by='timestamp')

    # Aggregating by hour
```

```
    df['hour'] = df['timestamp'].dt.floor('h')
    aggregated_df = df.groupby('hour').agg({'temperature':
'mean'}).reset_index()

    if len(aggregated_df) < 5:  # Adjust this value based on your data
availability
        logging.warning("Not enough data to predict.")
        return None

    # Prediction Logic
    aggregated_df['hour_index'] = range(len(aggregated_df))
    model = LinearRegression()
    model.fit(aggregated_df[['hour_index']], aggregated_df['temperature'])

    future_hours = np.array(range(len(aggregated_df), len(aggregated_df) +
24)).reshape(-1, 1)
    predictions = model.predict(future_hours)

    start_time = aggregated_df['hour'].iloc[-1] + timedelta(hours=1)
    future_timestamps = [start_time + timedelta(hours=i) for i in range(24)]

    result = [{'timestamp': ts.isoformat(), 'predicted_temperature':
round(temp, 2)} for ts, temp in zip(future_timestamps, predictions)]
    return result

def fetch_all_data_from_cosmos_db():
    # Query Cosmos DB to fetch all documents
    query = "SELECT * FROM c"  # Adjust this query based on your container
schema
    items = list(container.query_items(query=query,
enable_cross_partition_query=True))
    return items

def save_predictions_to_blob(predictions):
    # Convert predictions to DataFrame and save to CSV
    predictions_df = pd.DataFrame(predictions)
    predictions_csv = predictions_df.to_csv(index=False)

    # Create a unique filename based on current timestamp
    timestamp_str = datetime.now().strftime('%Y-%m-%dT%H-%M-%S')
    blob_name = f"predictions_{timestamp_str}.csv"

    # Upload the CSV to Blob Storage
    blob_client = container_client.get_blob_client(blob_name)
    blob_client.upload_blob(predictions_csv, overwrite=True)
    logging.info(f"Predictions saved to Blob Storage as {blob_name}")
```

```
def main(documents: func.DocumentList) -> None:
    logging.info(f"Cosmos DB Trigger fired with {len(documents)} documents.")

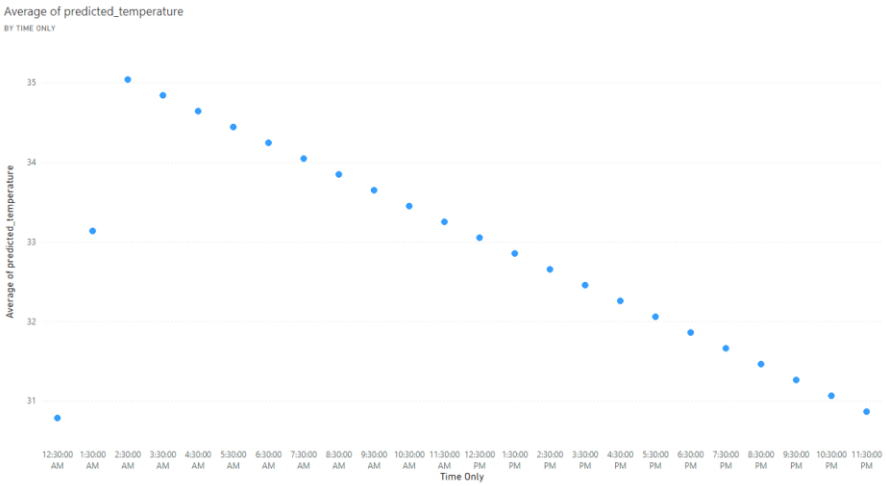
    # Fetch all data from Cosmos DB
    data = fetch_all_data_from_cosmos_db()

    if data:
        predictions = run_prediction(data)
        if predictions:
            logging.info("Predicted next 24 hours of temperatures:")
            for pred in predictions:
                logging.info(pred)

            # Save predictions to Blob Storage
            save_predictions_to_blob(predictions)

        else:
            logging.warning("Prediction could not be completed.")
    else:
        logging.warning("No data available for prediction.")
```

Data Visualization.



Predictive model – Linear Regression.

To enhance the intelligence and automation of the system, a temperature forecasting model based on linear regression is implemented using a Cosmos DB-triggered Azure Function App. This function is automatically triggered whenever new temperature data is written to Azure Cosmos DB, ensuring that forecasts are updated in near real-time. The model analyzes the historical temperature readings stored in the database and applies a linear regression algorithm to predict temperature values for the next 24 hours. These predictions are then seamlessly pushed to Power BI, where they are displayed through clear and interactive visualizations. By automating the prediction pipeline from

data ingestion to visualization, this approach enables users to anticipate indoor temperature changes and respond proactively, enhancing comfort and energy efficiency in a smart, connected environment.

4. Member Contribution.

Student ID	Proportion of Contribution
IT21174230	Gas Detection and Analysis
IT21380914	Temperature and Humidity Analysis

5. List Of Hardware.

I.Sensors.

- MQ2 Multi-Gas Sensor.
- DHT22 Temperature and Humidity sensor.

II.Actuators.

- 5V Relay Module.
- Diffuser.

III.Other Devices.

- 2 ESP32 Microcontrollers.
- Power supply : USB cables

6. Link to Code repository

https://github.com/IOTBDA2025/iot-data-collection-and-analysis-project-2025_2

7. Link to Video

https://mysliit-my.sharepoint.com/:v:/g/personal/it21174230_my_sliit_lk/Efosvfd4TO9MiwDDSbf7vmcBiRygxpyV9gUINLjAaOgk-w?nav=eyJyZWZlcnJhbEluZm8iOnsicmVmZXJyYWxBcHAiOiJPbmVEcml2ZUZvckJ1c2luZXNzIiwicmVmZXJyYWxBcHBQbGF0Zm9ybSI6IldlYiIsInJlZmVycmFsTW9kZSI6InZpZXciLCJyZWZlcnJhbFZpZXciOiJNeUZpbGVzTGlua0NvcHkifX0&e=90Y0wA